

OPERATING SYSTEMS PROJECT PROPOSAL

Semester: Spring 2026

1. PROJECT TITLE

Smart Parking Allocation System using POSIX Threads and Synchronization Mechanisms

2. TEAM INFORMATION

Project ID: 12

Section: BCS 4H

Faculty Name: Sir Zulfiqar Ali Rafique

Roll Number	Name	Contribution Area
24K 0977	Muhammad Abdullah	To Be Decided
24K 0987	Jiyanshu	To Be Decided
24K 0737	Yash Raj	To Be Decided

3. INTRODUCTION

The Smart Parking Allocation System is a concurrent simulation of a real-world intelligent parking management system implemented in C on a Linux (Ubuntu) environment. The system models vehicles as concurrent threads that arrive continuously and compete for a limited number of parking slots, which act as shared resources. The allocation and release of these slots are controlled using counting semaphores and mutex locks to ensure safe synchronization and prevent race conditions.

This project is highly relevant to core Operating System concepts such as concurrency, synchronization, resource allocation, scheduling, starvation prevention, and bounded waiting. The system mirrors real-world smart-city parking infrastructures where multiple vehicles simultaneously request limited parking spaces. By incorporating priority-based allocation (VIP vehicles), aging mechanisms, timeout handling, and performance statistics, the project demonstrates advanced OS principles including fairness policies, starvation avoidance, and system performance monitoring.

4. OBJECTIVES

Main Objective:

To design and implement a concurrent Smart Parking Allocation System in C using POSIX threads, counting semaphores, and mutex locks, while demonstrating advanced operating system synchronization and scheduling concepts.

Specific Objectives:

- Simulate vehicles as concurrent POSIX threads.
- Control limited parking slots using counting semaphores.
- Protect shared parking slot tables using mutex locks.
- Implement priority-based parking allocation (VIP vehicles).
- Prevent starvation using an aging mechanism.
- Handle extreme congestion through timeout handling.
- Analyze system performance using waiting time, throughput, and utilization metrics.
- Provide a real-time graphical dashboard using the ncurses library.

5. OPERATING SYSTEM CONCEPTS COVERED

Concept	Implementation
Concurrency	Multiple vehicle threads created using pthread_create()
Thread Management	Lifecycle control using pthread_join() and detached threads
Synchronization	Mutex locks to protect critical sections
Counting Semaphores	sem_init(), sem_wait(), sem_post() to manage slot availability
Critical Section	Parking slot allocation and queue modification
Scheduling	Priority based allocation (VIP vs Normal vehicles)
Starvation	Demonstrated when VIP vehicles dominate
Aging	Dynamic property upgrade after waiting threshold exceeded
Bounded Waiting	Timeout mechanism for extreme congestion
Resource Allocation	Parking slots treated as shared limited resources
Performance Evaluation	Waiting time, throughput, utilization statistics

6. METHODOLOGY

6.1 Programming Language & Tools

- Language: C
- Compiler: gcc
- Development Environment: Ubuntu Linux
- Libraries:
 - pthread (POSIX threads)
 - Semaphore.h
 - ncurses (for graphical dashboard)
 - ncurses (for graphical dashboard)
- Version Control: GitHub

6.2 Core Implementation Approach

Feature 1: Vehicle Threads

A separate thread will be created for each arriving vehicle. Vehicles will:

1. Attempt to enter the waiting queue.
2. Wait on a counting semaphore representing available parking slots.
3. Enter a critical section protected by a mutex to allocate a free slot.
4. Simulate parking for a random duration.
5. Release the slot safely and update statistics.

Feature 2: Semaphore for Parking Slots

A counting semaphore will be initialized with the total number of parking slots (20).

- `sem_wait()` will block vehicles when slots are full.
- `sem_post()` will increment availability upon exit.

This ensures that no more than 20 vehicles are parked simultaneously.

Feature 3: Mutex-Protected Slot Allocation Table

A shared array structure representing parking slots will be protected using a mutex to prevent:

- Double allocation
- Inconsistent slot states
- Race conditions during release

Feature 4: Safe Entry and Exit Handling

All modifications to:

- Parking slot table
- Waiting queue
- Statistics counters

Will be performed inside mutex-protected critical sections to ensure thread safety.

6.3 Additional Features Planned

Feature	Description
Priority Parking (VIP - 20%)	20% of vehicles will be marked VIP and receive priority allocation
Aging MEchanism	Vehicles waiting more that 15 seconds will have increased priority
Parking Timeout	Vehicles waiting beyond 30 seconds will exit the system
Waiting Queue Virtualization	Live display of current waiting vehicles in dashboard
Slot Release Logging	Log file recording entry, exit, waiting time and slot number
Utilization Statistics	Real time calculation of throughput and occupancy rate
Continuous Arrival Generator	Separate thread generating vehicles during a 120-second simulation

7. SYSTEM ARCHITECTURE / DESIGN

The system consists of the following major components:

- 1. **Vehicle Generator Thread** – Continuously creates vehicle threads until simulation time (120 seconds) ends.
- 2. **Vehicle Threads** – Represent cars requesting and releasing parking slots.
- 3. **Waiting Queue Module** – Circular queue structure protected by mutex.
- 4. **Semaphore Controller** – Limits total concurrent parked vehicles.
- 5. **Parking Slot Table** – Shared array representing 20 parking slots.
- 6. **Aging Monitor Thread** – Periodically scans waiting queue to upgrade priority.
- 7. **Statistics Module** – Computes system performance metrics.
- 8. **GUI Dashboard Thread** – Displays live parking layout and statistics using ncurses.

Data Flow Description:

Vehicle arrival → Waiting Queue → Semaphore Control → Mutex-Protected Allocation → Parking → Release → Statistics Update → GUI Display

The system runs for a fixed duration of **120 seconds**, after which final statistics are displayed.

8. DATA STRUCTURES USED

Data Structure	Purpose	Synchronization Mechanism
Array (Parking Slots)	Track occupied/free slots	Mutex
Circular Queue	Store waiting vehicles	Mutex
Structure (Vehicle)	Stores ID, type, timestamp	Protected via queue mutex
Global Counters	Statistics tracking	Mutex
Semaphore	Control slot availability	Counting semaphore

9. INTERFACE

Type: Graphical Terminal Dashboard using ncurses library

Description:

The system will provide a real-time graphical dashboard displayed in the terminal using the ncurses library. The interface will visually represent:

- A 20-slot parking grid (4×5 layout)
- Color-coded slot states:
 - Green: free
 - Red: occupied
 - Blue: VIP vehicle (
- Current waiting queue
- Total vehicles arrived
- Served vehicles
- Rejected vehicles
- Timeout vehicles
- Vehicles promoted via aging
- System utilization percentage
- Remaining simulation time

The display will refresh every second to simulate a live smart parking control system.

Key Features of Interface:

- Real-time parking slot visualization
- Live waiting queue monitoring
- Continuous statistics updates
- Clear color-coded state representation

10. TESTING PLAN

Test Case	Input Scenario	Expected Output	OS Concept Tested
1	High arrival rate	Queue grows but no double allocation	Semaphore correctness
2	VIP burst arrival	VIP served first	Priority scheduling
3	Long waiting normal vehicle	Priority upgraded	Aging mechanism
4	Extreme congestion	Some vehicles timeout and exit	Bounded waiting
5	Full capacity	sem_wait() blocks threads	synchronization
6	Rapid entry/exit	No inconsistent slot state	Mutex protection

11. TIMELINE / MILESTONES

Week	Deliverable	Status
Week 7	Proposal submission	Complete
Week 8-9	Environment Setup and basic thread structure	Incomplete
Week 10-11	Core synchronization implementation	Incomplete
Week 12-13	Priority, aging, timeout features	Incomplete
Week 14	GUI development	Incomplete
Week 15	Testing, debugging, and performance analysis	Incomplete
Final week	Demonstration, evaluation, and viva	Incomplete

12. CHALLENGES EXPECTED

- Debugging race conditions in multi-threaded environment
- Ensuring deadlock-free synchronization design
- Correct implementation of aging without priority inversion
- Managing dynamic thread creation efficiently
- Integrating real-time GUI updates with concurrent threads
- Accurate performance metric calculations

13. REFERENCES

- Silberschatz, Galvin & Gagne – Operating System Concepts
- Linux Man Pages (man pthread, man sem_init)
- GNU GCC Documentation
- ncurses Official Documentation
- POSIX Threads Programming Guide

14. GITHUB REPOSITORY

Repository Link: (To be created after approval)

All commits will reflect incremental development with proper version control practices.

15. DECLARATION

We hereby declare that this project proposal is our original work and will be implemented by us. We understand that the same project is not allowed within our section and approval is on a first-come-first-serve basis

Name	Signature	Date
Muhammad Abdullah	Abdullah	3/3/2026
Jiyanshu	Jiyanshu	3/3/2026
Yash Raj	Yash	3/3/2026

FOR TEACHER/EVALUATOR USE ONLY

Parameter	Remarks	Approval Status
Project Uniqueness		Accepted
Feasibility		Rejected
OS Concepts Coverage		Modify
Team Composition		

Remarks: _____

Faculty Signature: _____ Date: _____

Tips for Filling This Proposal:

1. Be specific - Avoid vague statements; mention exact system calls, functions, and algorithms
2. Show OS understanding - Explicitly connect your project to OS concepts
3. Be realistic - Choose additional features you can actually complete
4. Language compliance - Remember: Only C or C++ is allowed (no Python)